

ASP CODING REPORT

Report on Queueing Simulation for Single Server First-In-First-Out

METHOD

To simulate the event, I implemented a discrete-event simulation in Python. The simulation logic was built around the arrival processes and service processes, which determine the departure processes.

For arrival generation, two types of interarrival time distributions were modeled and later compared.

1. Exponential (Poisson arrivals): Interarrival times were from an exponential distribution with mean $1/\lambda$.
2. Uniform arrivals: Interarrival times were uniformly distributed on $[0, 2/\lambda]$, also with a mean of $1/\lambda$, to test the effect of using a wrong model with less variability.

Other information includes:

1. Service times were modeled as exponential with a mean of $1/\mu$.
2. Arrival times (cumulative sum of interarrivals).
3. Service start times (max of arrival time and previous departure)
4. Service completion times
5. Waiting time (service start - arrival)
6. System time (waiting + service)

Function Roles (I utilized Python's functions to write the script for the simulation.)

1. **generate_poisson_arrivals(n_customers, lam):** To generate the customers' arrival times following the Poisson process, compute the cumulative sums to get absolute arrival times.
2. **generate_uniform_interarrivals(n_numbers, lam):** Generates interarrival times uniformly distributed on $[0, 2/\lambda]$, ensures the mean interarrival time is still $1/\lambda$, but with lower variability than exponential, and creates an alternative (less variable) arrival process to test the wrong-model effect.
3. **generate_uniform_arrivals(n_customers, lam):** generates cumulative arrival times.
4. **generate_service_time(n_customers, mu):** Creates random service times for each customer using an exponential distribution. This represents how long each customer spends being served.
5. **simulate_single_server_fifo(arrival_times, service_times):** Simulates a queue with one server and first-come, first-served order. It calculates when each customer starts service, when they finish, how long they wait, and how much total time they spend in the system.

6. **theoretical_utilization(lam, mu)**: Calculates the theoretical utilization (traffic intensity) of a single-server queue, which would be used to plot the graph.
7. **actual_utilization(arrival_times, service_times, customer_departure_time)**: Generates the actual utilization of a single-server queue from simulation data by dividing the total service time by the simulation time horizon (from first arrival to last departure), real-world observation.
8. **Lq_littles_law(lam, average_wait_time)**: Generates the theoretical average number of customers waiting in the queue (L_q) using Little's Law by multiplying the arrival rate (λ) by the average waiting time. This approximation works well for steady-state queueing systems.
9. **Lq_discrete_events(arrival_times, service_start_times, customer_departure_times)**: Calculates the actual average number of customers waiting in the queue (L_q) by tracking discrete queue length changes throughout the simulation. It uses time-weighted averaging to account for how long the queue was at each length, providing a precise measurement of average queue size over the entire simulation period.
10. **run_grid(pairs=((0.5,1.0), (0.7,1.0), (0.9,1.0), (0.95,1.0)), N=2000,arrival_type="exp")**: This runs multiple queuing simulations across different arrival rate (λ) and service rate (μ) combinations. For each parameter pair, it generates arrivals (exponential or uniform), simulates the queue, and calculates both theoretical and actual performance metrics.

RESULTS

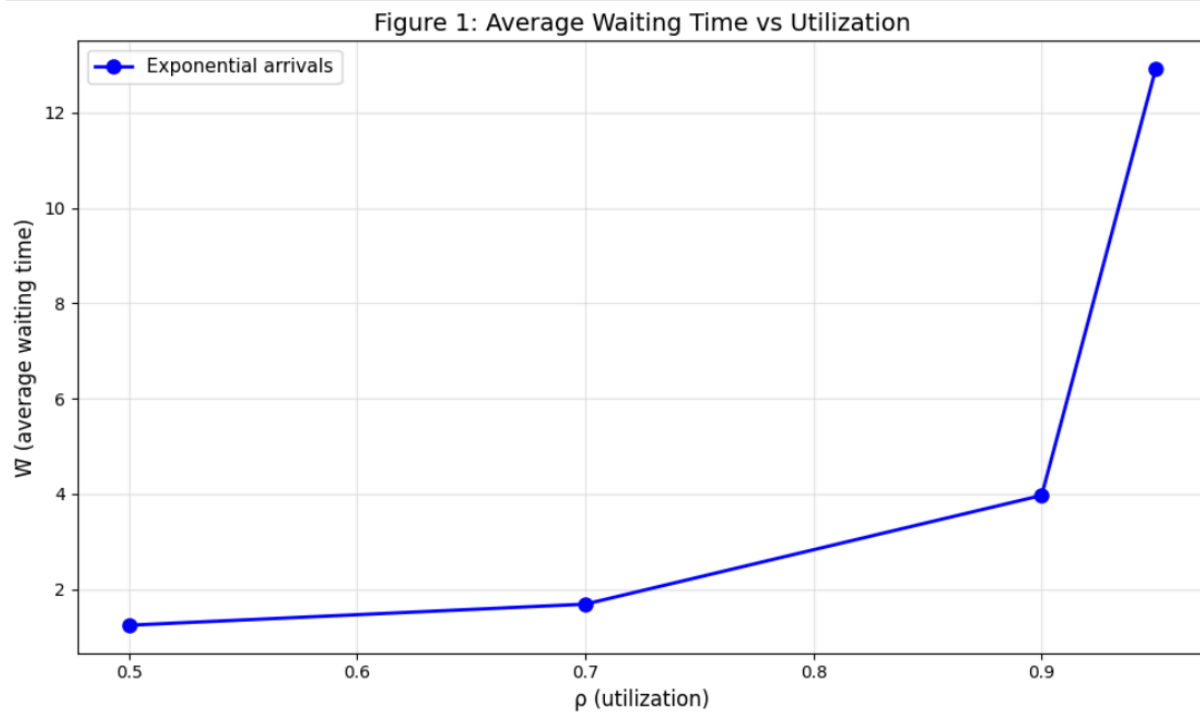


Figure 1: Average waiting time plotted against Utilization (traffic intensity) for Poisson arrival

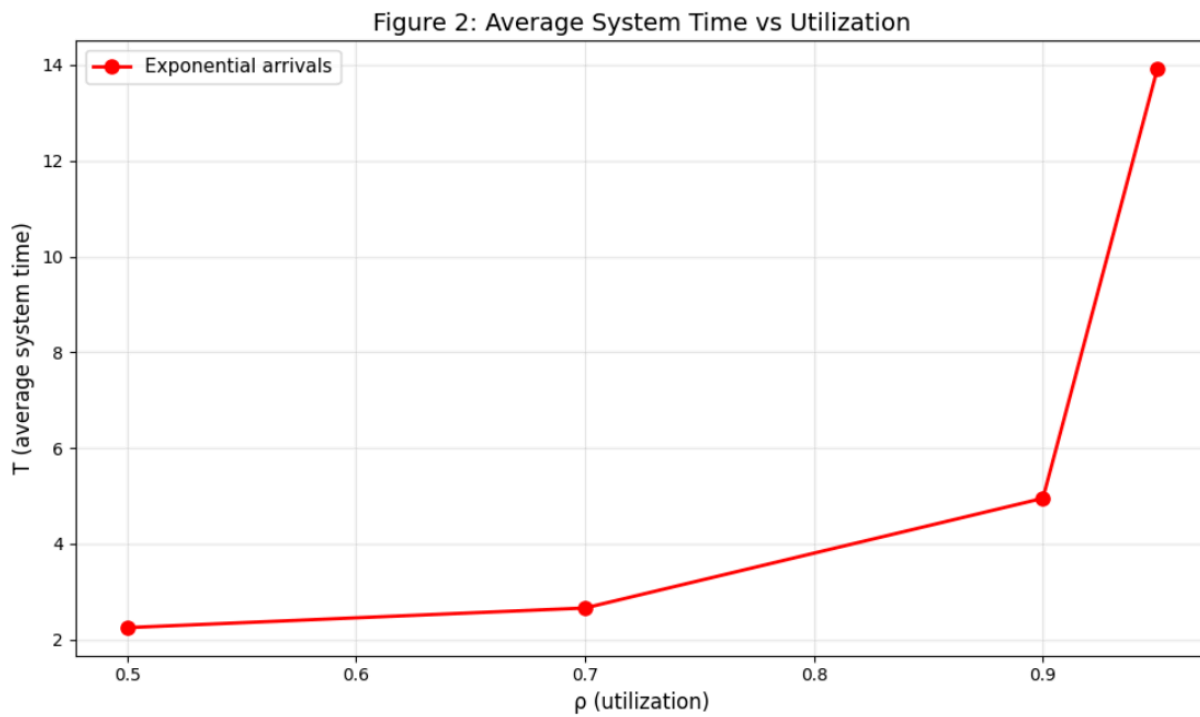


Figure 2: Average system time (waiting + service) against Utilization for Poisson arrival

Table 1: Exponential Interarrivals Summary

λ rate	μ rate	ρ theor	ρ actual	Average wait time	Average system time	Little's Law	Average queue length
0.5000	1.0000	0.5000	0.5057	1.2433	2.2472	0.6216	0.9694
0.7000	1.0000	0.7000	0.6635	1.6855	2.6576	1.1799	1.6880
0.9000	1.0000	0.9000	0.8323	3.9683	4.9489	3.5715	4.1490
0.9500	1.0000	0.9500	0.9327	12.9213	13.9211	12.2752	12.9451

Table 1: Summary of mean waiting times and system times across different values of utilization and average queue length for an exponential arrival.

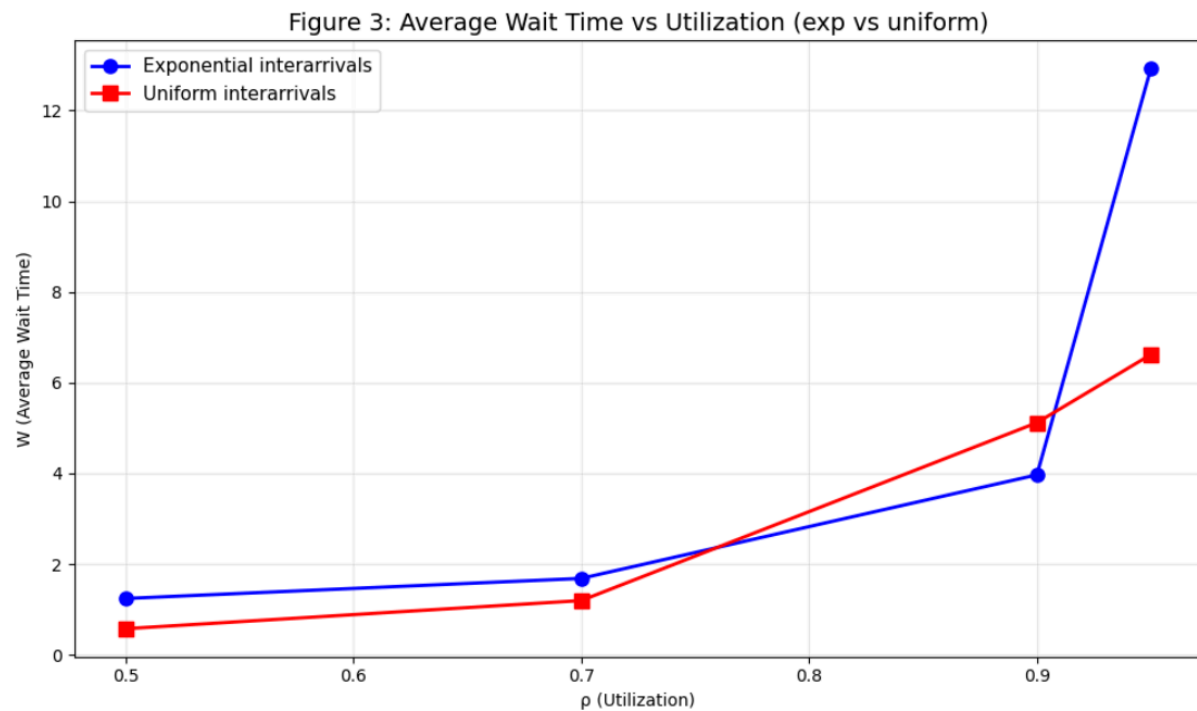


Figure 3: Average system time (waiting + service) versus Utilization under both Poisson and uniform arrivals.

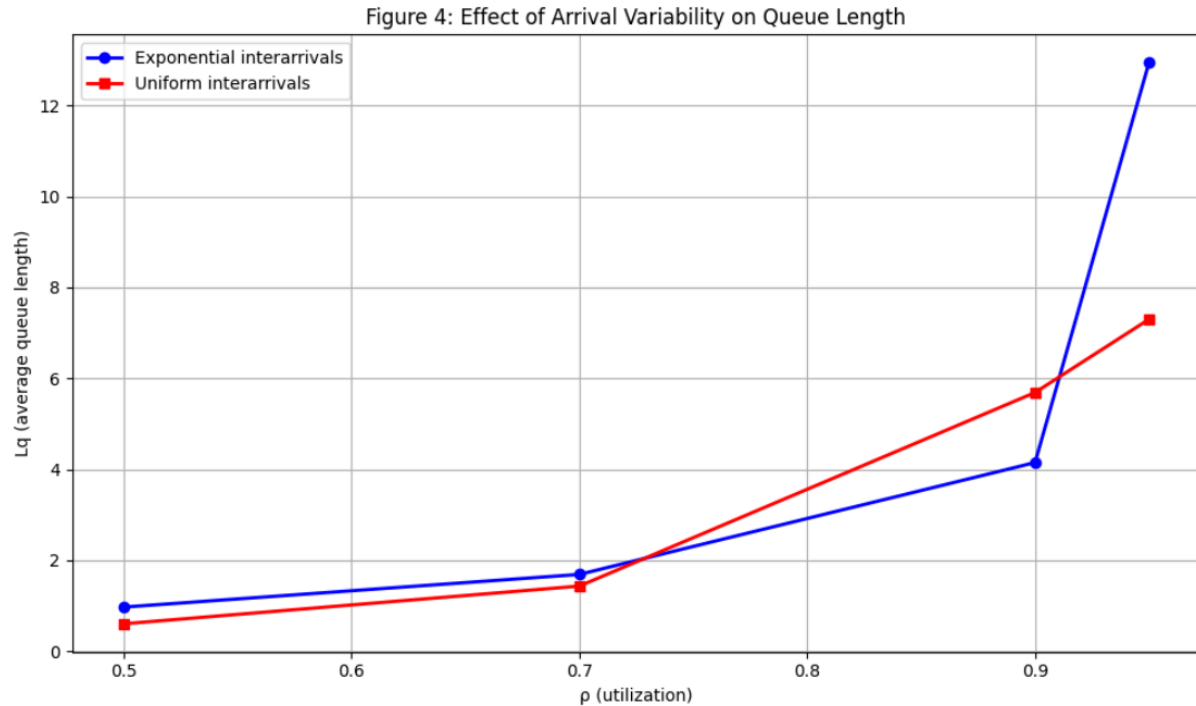


Figure 4: Sample queue length trajectories over time for Poisson and uniform arrivals. Plotting Average queue length against Utilization for both Poisson and Uniform arrivals.

Short Discussion: Impact of Arrival Variability on System Design

The comparison between exponential and uniform interarrivals with identical means reveals that arrival variability has a significant impact on queueing system performance, particularly at high utilization levels. Exponential interarrivals create random clustering where customers arrive in unpredictable bursts followed by gaps, leading to congestion and longer queues, while uniform interarrivals provide perfectly predictable spacing that eliminates clustering effects and maintains smoother system operation. As shown in Figure 4, this difference becomes increasingly critical as utilization approaches 1, where exponential arrivals produce nearly double the queue length ($L_q \approx 14$) compared to uniform arrivals ($L_q \approx 7.5$) at $\rho = 0.95$.

The "wrong modeling" effect demonstrates a dangerous system design pitfall: if engineers assume low-variability uniform arrivals when the true process follows high-variability exponential arrivals, they will severely underestimate the required queue capacity and service levels needed for acceptable performance. At high utilization levels, this modeling error becomes catastrophic, as a system designed for uniform arrivals would require almost double the queue space and buffer capacity to handle the reality of exponential arrival clustering. Such wrong assumptions could lead to system overloads, unacceptable customer wait times, and service failures in real-world applications where arrival variability is an inherent characteristic of the demand process.

Utilization as the Primary Performance Driver

The experimental results reveal the fundamental queueing theory principle that utilization ($\rho = \lambda/\mu$) is the primary determinant of system delay, with performance degrading non-linearly as ρ approaches unity. At moderate utilization levels ($\rho \leq 0.7$), both arrival processes demonstrate manageable performance with relatively low waiting times and queue lengths, indicating that systems with adequate spare capacity can absorb variability without significant performance penalties. However, as utilization increases beyond 0.9, the system enters a critical region where small increases in arrival rate or decreases in service capacity produce exponentially growing delays and queue lengths. This non-linear "performance cliff" effect means that high-utilization system design requires careful capacity planning with substantial safety margins, as the difference between acceptable and catastrophic performance can be determined by seemingly minor changes in system parameters.

The Critical Impact of Arrival Variability Assumptions

The comparison between exponential and uniform interarrivals with identical means ($1/\lambda$) exposes how arrival variability assumptions can fundamentally mislead system design decisions, particularly in high-utilization regimes. While both processes maintain the same theoretical utilization ($\rho = \lambda/\mu$), the exponential arrivals consistently produce significantly higher queue lengths due to their inherent clustering and burstiness, with the performance gap widening dramatically as utilization approaches capacity limits. At $\rho = 0.95$, the exponential arrivals generate nearly double the queue length ($L_q \approx 14$) compared to uniform arrivals ($L_q \approx 7.5$), demonstrating that wrong modeling assumptions can lead to catastrophic under-provisioning of system resources. For system designers, this variability effect represents a critical failure mode where assuming predictable, low-variability demand patterns when reality involves high-variability stochastic processes results in inadequate queue capacity, insufficient service levels, and potential system breakdown when the designed system encounters the true clustering nature of real-world arrival streams.

Real-World Design Implications

These findings have profound implications for practical system design across diverse applications, from telecommunications networks and computer systems to service operations and manufacturing processes. The wrong modeling effect suggests that system designers must carefully characterize the true variability of their demand processes rather than assuming simplified, convenient distributions that may underestimate performance requirements. In mission-critical applications such as emergency services, financial trading systems, or healthcare facilities, the cost of under-provisioning due to wrong variability assumptions can be catastrophic, making robust capacity planning with appropriate safety margins essential for reliable system operation under realistic demand conditions.